

ServiceOps complete platform manual

Version 1.0 · JNX branded edition

1. Purpose and operating model

ServiceOps is an enterprise service-management platform for requesters, fulfillers, team managers, CCB members, and platform administrators. It includes incident, request, problem, change, catalog, knowledge, CMDB, asset, SLA, approval, audit, analytics, and enterprise workspaces.

No deployment mechanism can eliminate every infrastructure or operator failure. The ServiceOps production standard therefore uses prevention, validation, observability, tested recovery, least privilege, immutable releases, and documented rollback rather than claiming infallibility.

2. Roles and teams

Role · Normal responsibilities

Requester · Submit and track requests and incidents, search knowledge

Agent · Triage, assign, fulfill, document, and resolve operational work

Manager · Agent work plus team oversight and manager approvals

CCB member · Review planned changes, risk, evidence, schedule, and backout plan

Administrator · Identity, ITIL configuration, CMDB, audit, and platform operation

CoreApps, Database, Network, Windows, Unix, and SSD are standard fulfillment teams. Every team manager is a CCB member. Normal and emergency changes follow manager assessment and CCB authorization.

3. End-user guide

Sign in

The sign-in page offers the identity methods enabled by the administrator: Keycloak SSO, AD/LDAP, and local administrator. The local administrator is a break-glass account and should not be used for routine work.

Navigation

The left application navigator contains role-appropriate modules. The top bar contains global search, favorites, recent history, notifications, help, and preferences. Use Preferences to select theme, density, font scale, contrast, reduced motion, navigation behavior, and start page.

Incidents

Create an incident with a concise title, observable symptoms, business impact, category, and urgency. Agents classify, prioritize, assign, comment, attach evidence, execute checklists, observe SLA timers, resolve, and close it.

Requests and catalog

Select a catalog item, supply its variables and business justification, and submit. ServiceOps creates REQ, RITM, approvals, and SCTASK records. Follow the request page for approval and fulfillment status.

Changes

Every change must state type, affected CI, owning team, risk, impact, planned window, implementation plan, test plan, and executable backout plan. Review conflicts before approval. CCB approval is authorization, not a substitute for technical validation.

Knowledge, CMDB, assets, and boards

Search knowledge before opening duplicate work. Use the CMDB relationship view to understand service impact. Asset pages track accountable inventory. The visual task board changes the underlying ticket state and therefore remains audited and role controlled.

4. Deployment decision

Environment · Application · Database · Upload storage

Laptop/demo · Docker Compose · Bundled PostgreSQL · Docker volume

Single production server · Docker Compose behind HTTPS proxy · Bundled or external PostgreSQL · Docker volume with backups

Enterprise production · Kubernetes/Helm, 3+ replicas · Managed HA PostgreSQL · RWX CSI volume or object-storage extension

Production Kubernetes must use an external highly available PostgreSQL service. The chart's bundled StatefulSet is deliberately limited to Demo/evaluation.

5. Docker installation

Run `./web-install.sh`, open `http://127.0.0.1:8090`, configure the profile, database, identity providers, and listener, validate all checks, confirm, and deploy. Keep the terminal open until the browser reports a healthy deployment.

Use `./serviceopsctl status`, `health`, `logs`, `backup`, `restore`, and `doctor` for lifecycle operations. Put Caddy, NGINX, or an enterprise load balancer in front of the loopback-bound application and terminate TLS there.

6. Kubernetes prerequisites

- Kubernetes 1.27 or newer with at least three worker nodes for HA.
- Helm 3, kubectl, a default-deny-capable CNI, and a CSI storage provider.
- A private image registry and immutable ServiceOps image tag or digest.
- External HA PostgreSQL with TLS, automated backups, and point-in-time recovery.
- RWX upload storage when application replicas exceed one.
- An ingress controller, DNS, and automated TLS certificate management.
- Metrics Server for HPA; cluster monitoring and log aggregation.
- A secrets controller or external vault for steady-state secret rotation.

7. Kubernetes installation

1. Build, scan, sign, and push an immutable image.

2. Copy ``deploy/kubernetes/values-production.example.yaml`` to

``deploy/kubernetes/values-production.yaml``.

3. Set registry, immutable tag, ingress, storage class, replicas, identity endpoints, and role mappings.

4. Run ``./kubernetes-install.sh --preflight``.

5. Run ``./kubernetes-install.sh``.

6. Confirm rollout, Helm test, ingress TLS, login, record creation, upload, approval, notification, and audit behavior.

The installer labels the namespace for Restricted Pod Security, creates secrets from a protected temporary file, uses ``helm upgrade --install --atomic --wait``, waits for rollout, and runs the packaged ``/ready`` test.

8. Kubernetes chart controls

- Non-root UID/GID, RuntimeDefault seccomp, all capabilities dropped.
- Read-only root filesystem and disabled service-account-token mounts.
- Startup, readiness, and liveness probes with distinct semantics.
- Rolling update with zero unavailable replicas.
- Topology spreading across zones and hosts.
- PodDisruptionBudget and optional HPA.
- Resource requests and limits.
- NetworkPolicy for ingress and required egress ports.
- Persistent upload claim and optional ingress/TLS.
- JSON schema validation and a Helm test hook.

Tune topology keys to the labels present in the target cluster. A PDB protects only against voluntary disruptions; it does not protect against node, zone, or application failure.

9. Identity configuration

Local administrator

Keep local authentication enabled for one vaulted break-glass administrator. Test it quarterly under an approved access procedure. Rotate after every use.

AD/LDAP

Use LDAPS or StartTLS, certificate validation, a least-privilege read-only bind identity, a narrow base DN, an escaped username filter, and explicit group-role mappings. Validate bind, search, user bind, disabled-user behavior, group mapping, certificate expiry, and directory outage behavior.

Keycloak

Use a confidential OIDC client, authorization-code flow, exact HTTPS redirect URI, short-lived tokens, approved realm-role claims, client-secret rotation, and restrictive redirect/web-origin settings. Validate login, logout, expired session, revoked user, missing email, role changes, and provider outage.

10. Security operations

- Enforce TLS externally and enable HSTS only after HTTPS is proven.
- Store secrets in a vault; never commit `.env`, values secrets, or kubeconfig.
- Restrict namespace RBAC and database privileges.
- Scan source, dependencies, image, manifests, and exposed endpoints in CI.
- Sign images and enforce admission verification where supported.
- Forward application, ingress, Kubernetes audit, database, and identity logs.
- Alert on repeated login failures, admin changes, approval anomalies, SLA

breaches, crash loops, readiness loss, saturation, and storage pressure.

- Patch base images and dependencies under change control.

11. Backup and recovery

Back up PostgreSQL and uploads as one recovery set. Encrypt backups, store them outside the cluster, apply retention and immutability, and record restore test evidence. For managed PostgreSQL enable PITR. Snapshotting a running database volume alone is not a proven logical backup.

Quarterly recovery test:

1. Restore into an isolated environment.
2. Restore database to the selected recovery point.
3. Restore uploads from the matching recovery set.
4. Deploy the matching immutable application version.
5. Validate counts, attachments, identities, approvals, audit, and critical

workflows.

6. Record actual RPO/RTO and corrective actions.

12. Upgrades and rollback

Back up first. Review release notes and schema compatibility. Render and lint the Helm chart, deploy to staging, run workflow regression and load tests, then use ``helm upgrade --install --atomic --wait``. Observe error rate, latency, readiness, database health, and queues. ``--atomic`` rolls Kubernetes resources back when the upgrade fails, but database schema/data rollback still requires a release-specific tested procedure.

13. Monitoring and SLOs

Monitor availability, request rate, latency percentiles, HTTP errors, worker saturation, pod restarts, readiness, CPU/memory throttling, database connection usage, query latency, replication/backup health, volume capacity, login errors, notification failures, and SLA breach rate. Define business-approved SLOs and page only on actionable symptoms.

14. Incident response

Declare severity and incident commander, preserve evidence, stabilize service, communicate on a fixed cadence, use tested rollback/failover, validate recovery, and create a blameless problem record with corrective actions. Never delete

audit or operational evidence during response.

15. Production acceptance checklist

- Immutable, scanned image from trusted registry
- External HA PostgreSQL with TLS, PITR, and restore test
- Three or more application replicas across failure domains
- RWX persistent uploads and tested restore
- TLS ingress, DNS, HSTS, security headers
- Restricted Pod Security and least-privilege RBAC
- Network policies validated with the actual CNI and endpoint topology
- LDAP/Keycloak end-to-end tests and vaulted break-glass account
- Monitoring, logs, alert routing, dashboards, and runbooks
- Load, soak, failover, node-drain, rollback, and disaster-recovery tests
- Security review, penetration test, and formal go-live approval